

# Mock Test 1

Started: Aug 3 at 9:52am

## Quiz Instructions



### Question 1 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer previously analyzed a legacy authentication module and identified two distinct refactoring approaches. They now want to explore both in depth before deciding. What is the most effective way to structure this exploration?



Resume the old session for the first approach, then start a new one for the second.



Start two fresh sessions and manually summarize yesterday's findings into each.



Resume the old session and explore both approaches sequentially in one conversation.



Use `fork_session` to create two branches from the prior analysis and explore one approach in each fork.



### Question 2 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

When the coordinator agent calls a subagent using the Task tool, what is NOT automatically inherited by the subagent?



System prompt configuration



Tool definitions available to the agent



Parent session context and conversation history



User authentication credentials



### Question 3 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Your coordinator detects that one research subagent is making repeated incorrect tool calls. What is the most reliable way to enforce compliance at the architecture level?



Use a `PostToolUse` hook to validate tool calls before they execute



Reduce the number of tools available to that subagent via `allowedTools`



Modify the subagent's system prompt to include stricter rules



Add a validation step in the coordinator that rejects bad requests



### Question 4 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Your web search subagent needs to refine its queries based on initial results before synthesizing findings. How should you structure this in the agentic loop?



Create a prompt chain that executes all searches sequentially without feedback



Let the subagent iterate internally - it sees tool results, evaluates them, and calls the search tool again if needed



Have the coordinator re-invoke the subagent multiple times with updated prompts



Queue all queries in advance and execute them in parallel



### Question 5 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web,

one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Your synthesis subagent needs to work from research conducted by both the web search and document analysis subagents. What is the recommended pattern for providing this information?



Have the synthesis subagent invoke both the web search and document analysis tools directly



Have each subagent store results in a shared database that the synthesis agent queries



Pass the coordinator's summary of both subagents' findings in the Task tool call to the synthesis subagent



Chain subagent calls: first get results, then invoke synthesis with those results passed as context



### Question 6 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

You want the coordinator to detect when a subagent's response is incomplete and automatically trigger a refinement loop. Where should this logic be implemented?



In a separate validation microservice called after all subagents complete



In a PostToolUse hook attached to the Task tool



In the subagent's system prompt as conditional rules



In the coordinator's agentic loop - after receiving the subagent's result via the Task tool, the coordinator evaluates completeness and re-invokes if needed



### Question 7 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Your coordinator needs to route research tasks to subagents, but some tasks might be better served by multiple subagents. How should you structure this decision-making?

☐ Create a decision tree in `.claude/rules/` that lists all topic-to-subagent mappings

☐ Queue all possible subagent invocations and execute in parallel regardless of relevance

☐ Let the coordinator use model-driven reasoning to decide: based on the research question and available subagents, decide dynamically how to partition work

☐ Hard-code rules in the coordinator's system prompt: 'Topic X always goes to subagent A'



### Question 8 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Your coordinator implements a refinement loop: after evaluating findings, if gaps exist, re-invoke the subagent with specific gap prompts. How many re-invocation rounds should you allow before considering the research 'good enough'?

☐ Set a hard limit (e.g., max 3 rounds) to control costs and latency

☐ Use adaptive stopping: continue if new information is discovered, stop when returns diminish

☐ Unlimited; iterate until the subagent reports completion

☐ Re-invoke once per unique gap identified, then synthesize the result



### Question 9 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture across a large multi service codebase. What exploration strategy most effectively builds understanding under context limits?

☐ Launch parallel subagents to explore different services simultaneously and then synthesize their findings.

☐ Use Grep to find authentication entry points, read those files, and follow imports and function calls incrementally.



Read every file containing auth related keywords in content or filename.



Read CLAUDE.md and README files first, then ask the engineer to identify the most important files.



### Question 10 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Engineers often want the agent to cross reference code changes with Jira tickets during reviews. The team wants the agent to access standard Jira data directly rather than via manual copy paste. What is the most effective approach?



Export Jira tickets to markdown files in the repository.



Build a custom MCP server around Jira's API for this workflow.



Integrate an existing Jira MCP server that exposes tickets, comments, and metadata as tools.



Use Bash and curl directly against Jira's REST API.



### Question 11 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your code review assistant must evaluate pull requests for style, security, and documentation using the same three step workflow for every PR. Which task decomposition pattern is most appropriate?



Prompt chaining, with each aspect analyzed separately and combined in a final synthesis step.



A single comprehensive prompt that handles all three aspects simultaneously.



An orchestrator workers pattern that dynamically chooses which checks are needed.



A routing pattern that first classifies the PR by type.



## Question 12 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

After a long exploration of a rendering subsystem, the agent starts responding with generic rendering patterns instead of the specific classes it had discovered. An engineer now asks how physics integrates with rendering for debug overlays. What is the most effective approach?



Summarize the key rendering findings, then spawn a subagent for physics exploration with that summary in its initial context.



Clear the context completely and restart the exploration from scratch.



Spawn a subagent to explore physics independently and manually synthesize later.



Continue in the current context with more targeted prompts naming the specific classes.



## Question 13 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks the agent to find all callers of a function before removing it. The function is exposed through wrapper modules that rename it for domain specific use. What exploration strategy will most reliably identify all callers?



Search project documentation to understand intended usage patterns.



Find all files that import from the library or wrapper modules, then read them to check usage.



Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase.



Use Grep to search for the function's original name across the codebase.



## Question 14 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer wants to understand a caching layer before adding a new invalidation trigger. The caching logic spans many files. What is the most effective next step while managing context constraints?



Use Glob to find cache related filenames and read the largest files first.



Sequentially load all files with the Read tool.



Use Grep to search for invalidate and expire patterns and read only those line ranges.



Analyze imports and class hierarchies to identify the base cache class, read it first, then trace specific invalidation implementations.



Question 15 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent is exploring a large codebase and finds 150 matching files with a Glob pattern. To understand file relationships, what should it do next?



Ask the user which files are important



Use Bash to generate a dependency graph automatically



Use Read to open all 150 files and analyze them



Use Grep to search for import statements, function calls, and dependencies across the matched files



Question 16 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent encounters a large configuration file that the Bash 'head' tool would normally read in pieces. Should the agent use Read tool instead?

☐

No; always use Bash for efficiency

☐

No; large files should always be processed with Bash piping

☐

Use Read only if the file is less than 100 lines

☐

Yes; the Read tool is better for reading file contents, especially large files, because it provides line numbering and handles formatting



Question 17 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

When your agent reads a source file that hasn't been modified in 2 years, what strategy helps understand its purpose without reading all of it?

☐

Use Grep to find test files or documentation that reference this module

☐

Read the entire file to understand its full context

☐

Skip it; old code is usually not important

☐

Use Bash to count lines and estimate complexity



Question 18 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent needs to understand how a function is used throughout the codebase. What Grep pattern should it use?

☐

Search for the function name: 'grep functionName'

☐ Search for function calls: 'grep functionName(' or 'grep functionName\\('

☐ Search for imports that might use this function

☐ Try all three: search for the name, search for calls, search for imports



### Question 19 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent is generating boilerplate code (a database migration script) and writes it to a file using the Write tool. After writing, what should the agent do?

☐ Run Bash to execute the generated code and verify it works

☐ Ask the user to review the generated code before using it

☐ Read the file back to verify the generated content matches expectations

☐ Immediately return the file path to the user



### Question 20 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent encounters an Edit tool failure: 'old\_string is not unique in the file'. What's the best recovery strategy?

☐ Use Bash sed to make the change instead

☐ Try Edit again with a larger substring that includes more context

☐ Ask the user to manually edit the file

☐ Use Read to examine the file, then use Write with a complete file rewrite



### Question 21 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent needs to understand the dependency structure of a large codebase. What's the most efficient approach?



Read every file in the codebase



Use Glob to find file patterns (imports, module declarations) and Grep to map relationships



Visualize the entire codebase first with external tools



Use Bash to run build tools or language-specific analysis (gradle, npm, cargo)



### Question 22 15 pts

**Scenario: Code Generation with Claude Code** You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team has a monorepo with different services. You want to create a custom command that generates boilerplate for a new microservice. Where should this command be stored to ensure it's version-controlled and shared across the team?



.claude/microservice\_generator.md in the project



A dedicated /commands directory at the repository root



~/.claude/commands/ in the user's home directory



.claude/commands/ in the project root



### Question 23 15 pts

**Scenario: Customer Support Resolution Agent** You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get\_customer, lookup\_order, process\_refund, escalate\_to\_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your agent processes a customer return request and after calling `lookup_order`, the response arrives with `stop_reason` set to `'tool_use'`. What should you do next in the agentic loop?



Automatically send another request to Claude without executing any tools yet



Treat the `stop_reason` as a signal to end the conversation and return results to the user



Extract the tool name and input from the `stop_reason` field and execute immediately



Inspect the assistant's message for `tool_use` blocks, extract the tool name/input, execute the tool externally, and append results back to conversation history



Question 24 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers. When your codebase exploration agent uses the Glob tool to find files, what is the primary value of Glob over using Bash `'find'` command?



Glob can filter by file content, not just names



Glob works reliably across different operating systems with consistent pattern matching



Glob returns results sorted by modification time, making it easy to identify recently changed files



Glob is faster because it doesn't traverse directories



Question 25 15 pts

**Scenario: Multi-Agent Research System** — You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports. When evaluating synthesis quality, the coordinator determines that key research gaps exist.

What is the most efficient pattern for addressing gaps without restarting the entire research?



Queue gap queries and execute all in a single new coordinator invocation



Summarize findings to date and start a fresh session with the new research direction



Use `fork_session` to branch from the current research state and explore alternatives in parallel



Invoke the search and analysis subagents again with targeted prompts to fill specific gaps



### Question 26 15 pts

**Scenario: Customer Support Resolution Agent** You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your agent processes a customer return request and after calling `lookup_order`, the response arrives with `stop_reason` set to `'tool_use'`. What should you do next in the agentic loop?



Automatically send another request to Claude without executing any tools yet



Treat the `stop_reason` as a signal to end the conversation and return results to the user



Inspect the assistant's message for `tool_use` blocks, extract the tool name/input, execute the tool externally, and append results back to conversation history



Extract the tool name and input from the `stop_reason` field and execute immediately



### Question 27 15 pts

**Scenario: Customer Support Resolution Agent** You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

You've implemented a hub-and-spoke coordinator-subagent architecture where the main agent delegates billing dispute resolution to a specialized subagent using the Task tool. What critical configuration detail must you handle explicitly?



The Task tool creates a new session, but both agents continue using the same conversation object



The coordinator agent should block the subagent from calling any external tools to prevent conflicts



You must explicitly pass customer context and available tools to the subagent; context is not inherited automatically



The subagent automatically inherits the parent agent's conversation history and tool access



### Question 28 15 pts

Scenario: Customer Support Resolution Agent You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your agent receives a request about a package that wasn't delivered. To ensure it always gathers the order details before attempting a refund, you want to enforce a prerequisite: `get_customer` and `lookup_order` must complete before `process_refund` is callable. How should you implement this programmatically?



Restrict `process_refund` using `tool_choice`: 'any' to allow only that tool in the next turn



Write a detailed prompt asking the agent to 'always check customer details and order status before processing refunds'



Create a pre-configured decision tree in your orchestration code that routes to `get_customer`, then `lookup_order`, then `process_refund` sequentially



Implement a `PostToolUse` hook that validates whether `lookup_order` was called; if not, raise an error preventing `process_refund` from executing



### Question 29 15 pts

Scenario: Customer Support Resolution Agent You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

You're designing an escalation workflow. When the agent decides a customer issue needs human review, it should pass not just the customer ID but also the root cause and recommended next steps. What format best enables reliable handoff to the human support team?



Store the escalation details in a database separately and send the human agent a reference ID to look up the details



Call `escalate_to_human` with just the customer ID and let the human agent read the full conversation history



Create a text summary of the conversation and pass it as a single unstructured string to `escalate_to_human`



Implement a structured escalation protocol where `escalate_to_human` receives customer ID, issue summary, root cause, and recommended actions in defined fields



### Question 30 15 pts

**Scenario: Customer Support Resolution Agent** You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your agent is handling multiple customer issues in a single conversation: one customer is asking about a billing dispute AND a return. You decide to use task decomposition with the Task tool to parallelize work. What's a key advantage of this approach over a sequential single-agent flow?



Parallel subagents allow you to assign specialized agents to each issue (billing expert, returns expert) simultaneously, reducing latency and potentially catching issues that a generalist agent might miss



Task decomposition automatically merges results from parallel subagents without any manual logic



Using parallel tasks eliminates the need for the agent to understand context; each subagent works independently



Task decomposition guarantees that all issues are resolved in the same turn without additional conversation rounds



### Question 31 15 pts

**Scenario: Customer Support Resolution Agent** You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

You're resuming a support session from yesterday using the `--resume` flag in the Claude SDK. The agent had gathered customer details and identified a billing dispute. What's the risk of resuming a stale session versus starting fresh with a summary?



There's no meaningful difference; resume and fresh start produce identical agent behavior



Resuming is always faster and has no downsides compared to starting fresh



Resuming uses the old session's context window, potentially hitting limits faster if the conversation was long



Resuming guarantees the agent remembers the context, whereas a fresh session requires re-teaching the agent what happened



### Question 32 15 pts

Scenario: Customer Support Resolution Agent You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your agent's system prompt includes a rule: 'Always verify the customer identity before processing any refund.' This is critical for compliance. However, you're concerned the agent might skip this step if it feels confident about the identity from the conversation. What's the most reliable enforcement mechanism?



Create a separate verification agent that must sign off before any refund can proceed



Use `tool_choice`: 'required' to force the agent to always call `get_customer` first



Make the system prompt rule more detailed and explicit with multiple reminders about identity verification



Implement a `PostToolUse` hook that inspects whether `get_customer` (identity verification) was called before allowing `process_refund`



### Question 33 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, `CLAUDE.md` configurations, and understand when to use plan mode vs direct execution. You're making a complex refactor that touches 8 files and requires careful architectural decisions. The changes are interconnected and depend on understanding the current design patterns. Should you use plan mode or direct execution?



Use direct execution since Claude Code handles file dependencies automatically



Use direct execution for speed, since plan mode adds unnecessary overhead



Use plan mode to outline the refactoring strategy before making changes



Use plan mode only if the refactor affects more than 10 files



## Question 34 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. Your project has a .claude/CLAUDE.md file and a .claude/rules/ directory with several rule files. A developer works in the /services/auth directory. How does Claude Code determine which rules apply when editing files in that directory?



Only rules explicitly imported via @import in the project CLAUDE.md



Rules with glob patterns matching the file path, plus the nearest directory-level CLAUDE.md, plus project-level CLAUDE.md



All rules in .claude/rules/ are always applied regardless of file path



Rules are loaded in the order they appear in the .claude/rules/ directory



## Question 35 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. You've created a skill in .claude/skills/test\_generator/ with context: fork in the frontmatter. What does this configuration do?



The skill forks the project repository, creating a temporary branch for test generation



The skill runs in parallel with the main session, allowing concurrent operations



The skill runs in an isolated sub-agent with its own context, preventing pollution of the main Claude Code session



The skill executes in the current session context, sharing all files and previous interactions



## Question 36 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You want to create a skill that can read files but should NOT have access to write, edit, or execute commands. How do you restrict the skill's capabilities?



Add a no-write flag to the skill's frontmatter



Use allowed-tools in the skill's SKILL.md frontmatter to specify only read and glob tools



Create the skill in ~/.claude/skills/ instead of .claude/skills/



Skills inherit restrictions from CLAUDE.md; define rules in the project configuration



### Question 37 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. Your team runs Claude Code in a CI/CD pipeline using the -p flag. What does this flag enable?



Non-interactive mode for CI execution without requiring user prompts or approval



Plan-mode-only execution, forcing all changes to go through planning first



Parallel execution, allowing multiple Claude Code instances to run simultaneously



Performance profiling to measure execution time and resource usage



### Question 38 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. Your CLAUDE.md file is getting large and hard to manage. You want to split common standards into separate files while keeping the main CLAUDE.md concise. How do you reference these separate files?



Use @import syntax to modularly reference other CLAUDE.md or configuration files



Split the file manually and configure path matching in .claude/rules/



Create multiple CLAUDE.md files in different .claude/ subdirectories



Use include: directives in the CLAUDE.md header



### Question 39 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. Your project uses MCP servers for database access. Where should you store the .mcp.json configuration so it's shared with the team?



In a .mcp/ directory in the project root



In ~/.claude.json in each user's home directory



In the .claude/rules/ directory with path matching



In the project root or .claude/ directory to be version-controlled



### Question 40 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. .mcp.json contains database credentials. How should you handle credential storage securely?



Embed credentials directly in .mcp.json and restrict file permission



Use environment variable expansion in .mcp.json (e.g., \$DB\_PASSWORD) and store variables locally



Store credentials in .claude/secrets.json with additional encryption



Use a separate .env.local file that .mcp.json references



### Question 41 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You

need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. You're building a code review tool and need Claude Code to search for specific patterns in thousands of files. Which built-in tool is optimized for this task?



The Read tool, which can process multiple files sequentially



The Grep tool, which uses ripgrep for fast content pattern matching



The Bash tool with find and grep commands



The Glob tool combined with individual file reads



#### Question 42 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. You're designing a custom tool for your domain and want Claude Code to use it effectively. What is the most important factor in the tool definition?



A clear, descriptive tool description that helps the model understand when to use it



A list of example use cases in the tool name



Detailed parameter documentation with many optional fields



A complex implementation that shows the tool's full capabilities



#### Question 43 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. You have 5 custom tools but want Claude Code to use them predictably. You're getting unexpected tool selections. What's the best approach to control tool usage?



Run Claude Code with a simpler system prompt



Use tool\_choice to force selection of specific tools for specific tasks



List all tools in CLAUDE.md as required tools



Give all tools identical descriptions to avoid bias



#### Question 44 15 pts

Scenario: Code Generation with Claude Code You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution. Your MCP server returns structured errors with errorCategory and isRetryable fields. How should Claude Code handle a retryable error?



The errorCategory determines whether the error is retried



All errors should be treated identically regardless of the isRetryable flag



Retryable errors should trigger a skip to the next tool



The isRetryable flag indicates the tool call should be retried; Claude Code should attempt the operation again

---

Not saved

Submit Quiz